# Sesión 10 — Laboratorio

Programación Declarativa
3º del Grado en Ingeniería Informática
Funciones recursivas sobre listas

---

**Nota**

Abre GHCi y carga tus definiciones con `:load fichero.hs` (o `:l fichero.hs`). Recuerda que puedes recargar tras editar con `:reload` (o `:r`). Comprueba los tipos de tus funciones con `:type` (`:t`) antes de probarlas.
Los ejercicios marcados con ★ son los recomendados para completar durante la sesión si el tiempo es limitado.

## Parte A — Recursión básica sobre listas

### Ejercicio 1.

Define `miUnzip :: [(a,b)] -> ([a],[b])` que separe una lista de pares en un par de listas, **por recursión directa** sobre la lista de pares (sin usar `unzip`, ni `map fst`/`map snd`).

```
ghci> miUnzip [(1,'a'),(2,'b'),(3,'c')]
([1,2,3],"abc")
```

### Ejercicio 2.

Define `contarVocales :: String -> Int` que cuente las vocales de una cadena **por recursión explícita** (sin usar `filter` ni `length`).

```
ghci> contarVocales "Programacion Declarativa"
10
```

### Ejercicio 3.

Define `ultimaAparicion :: Eq a => a -> [a] -> Int` que devuelva la posición (empezando en 0) de la **última** aparición de un valor en una lista, o `-1` si no aparece.

```
ghci> ultimaAparicion 2 [1,2,3,2,4,2]
5
ghci> ultimaAparicion 9 [1,2,3]
-1
```

### Ejercicio 4. ★

Define `intercalar :: [a] -> [a] -> [a]` que combine dos listas alternando sus elementos uno a uno (a diferencia de `zip`, el resultado es una lista **plana**, no de pares). Si una lista se agota antes que la otra, el resto de la lista más larga se añade al final.

```
ghci> intercalar [1,3,5] [2,4,6]
[1,2,3,4,5,6]
ghci> intercalar [1,2] [10,20,30,40]
[1,10,2,20,30,40]
```

## Parte B — Particionar una lista

**Ejercicio 5.**

Define `particionar :: Ord a => a -> [a] -> ([a],[a],[a])` que divida una lista en tres, según sus elementos sean menores, iguales o mayores que un pivote dado, **por recursión** (sin usar `filter` tres veces).

```
ghci> particionar 5 [3,7,5,1,5,9,2]
([3,1,2],[5,5],[7,9])
```

**Ejercicio 6.** ★

Define `particionarPredicado :: (a -> Bool) -> [a] -> ([a],[a])` que divida una lista en dos según cumplan o no un predicado arbitrario.

```
ghci> particionarPredicado even [1,2,3,4,5,6]
([2,4,6],[1,3,5])
```

## Parte C — Inserción ordenada y comprobación de orden

**Ejercicio 7.**

Define `insertarDescendente :: Ord a => a -> [a] -> [a]` que inserte un elemento en una lista que está ordenada de **mayor a menor**, manteniendo el orden descendente.

```
ghci> insertarDescendente 5 [9,7,3,1]
[9,7,5,3,1]
```

**Ejercicio 8.**

Define `estaOrdenada :: Ord a => [a] -> Bool` que compruebe, por recursión, si una lista está ordenada de menor a mayor.

```
ghci> estaOrdenada [1,3,3,7]
True
ghci> estaOrdenada [1,5,2]
False
```

**Ejercicio 9.** ★

Define `numeroInversiones :: Ord a => [a] -> Int` que cuente cuántos pares de elementos están "fuera de orden": es decir, cuántas veces un elemento es **mayor** que otro elemento que aparece **después** que él en la lista. Es una medida de cuán desordenada está una lista: una lista ordenada tiene 0 inversiones.

```
ghci> numeroInversiones [1,2,3]
0
ghci> numeroInversiones [3,1,2]
2
```

## Parte D — Acumuladores y recursión de cola

**Ejercicio 10.** ★

Define `sumaAcc :: [Int] -> Int` usando un acumulador (función auxiliar de cola), y compara con `:set +s` su rendimiento frente a `sum` (o frente a una versión recursiva sin acumulador) sobre una lista grande, como `[1..2000000]`.

```
ghci> sumaAcc [1,2,3,4]
10
```

**Ejercicio 11.**

Define `maximoAcc :: Ord a => [a] -> a` que calcule el máximo de una lista **no vacía** usando un acumulador.

```
ghci> maximoAcc [3,7,2,9,4]
9
```

## Parte E — Combinando con funciones de orden superior

**Ejercicio 12.**

Define `runLength :: Eq a => [a] -> [(a,Int)]` que comprima una lista agrupando **rachas** de elementos consecutivos iguales, junto con su longitud (compresión por longitudes de racha, *run-length encoding*).

```
ghci> runLength "aaabbc"
[('a',3),('b',2),('c',1)]
ghci> runLength [1,1,2,2,2,3]
[(1,2),(2,3),(3,1)]
```

*Pista:* puedes resolverlo por recursión explícita, o usando `takeWhile`/`dropWhile` para separar la racha inicial del resto.

## Parte F — Ejercicio integrador: *mergesort*

**Ejercicio 13.** ★

Implementa la ordenación por fusión (*mergesort*) en tres pasos:

a) `dividir :: [a] -> ([a],[a])`, que separe una lista en dos mitades de tamaño lo más parecido posible

b) `fusionar :: Ord a => [a] -> [a] -> [a]`, que combine dos listas **ya ordenadas** en una sola lista ordenada

c) `mergesort :: Ord a => [a] -> [a]`, que divida la lista recursivamente, ordene cada mitad, y las fusione

```
ghci> mergesort [5,3,8,1,9,2]
[1,2,3,5,8,9]
```

Compara con quicksort (visto en teoría): ¿en qué se diferencia la estrategia de división, y por qué *mergesort* garantiza coste $O(n \log n)$ incluso en el peor caso, a diferencia de quicksort?